

Algoritmos y Estructuras de Datos

Problem Sets UT3 – Árbol Genérico y Tries.

Se debe subir un archivo zip con las siguientes características:

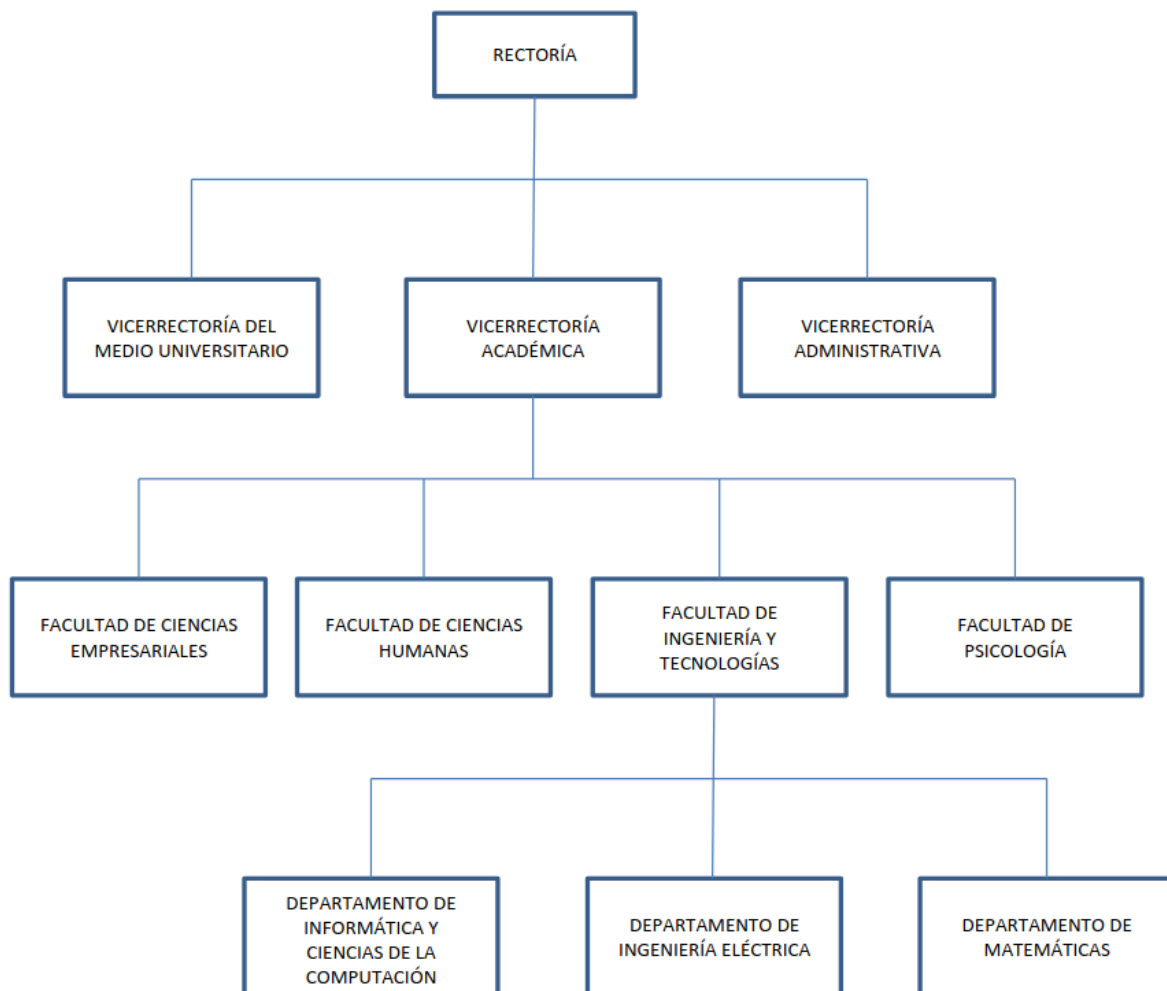
Nombre del archivo: `equipo-{numero de equipo}-{nombre de docente}.zip`

El mismo debe contener todo el código generado e imágenes o documentos en pdf en aquellos ejercicios que no se resuelven en código.

Ejercicios entregables para corrección: 5, 7, 9, 11, 12, 13, 14, 15, 16

Ejercicio 1

La Universidad Católica del Uruguay necesita desarrollar una aplicación informática que le permita mantener su organigrama. Un ejemplo del organigrama actual de la UCU es, en parte y en forma simplificada, el siguiente:



El analista del proyecto ha definido que la mejor forma de representar este problema es mediante el tipo de datos abstracto árbol. El árbol tiene una referencia a un nodo llamado raíz, y a su

vez los nodos tienen una referencia a su hermano derecho, una referencia a su primer hijo, una etiqueta que lo distingue y una referencia a donde están almacenados los datos de ese nodo.

De las funcionalidades que debe tener el sistema a desarrollar, el analista ha asignado a su grupo de desarrolladores las siguientes:

- Modificar los datos de un componente dado.

Para cumplir con los requerimientos, su equipo deberá desarrollar el método para recuperar un nodo del árbol. Desarrollar en pseudocódigo el siguiente algoritmo, con las firmas dadas:

- `buscar(Comparable<T>criterio)`: devuelve el nodo que satisface el criterio; en caso contrario devuelve el nodo nulo.

En cada uno de los casos debe escribirse el método del árbol y el método del nodo.

1. Descripción en lenguaje natural del algoritmo solicitado.
2. Identificación de precondiciones y postcondiciones correspondientes.
3. Escritura del algoritmo en pseudocódigo formal.
4. Análisis detallado del orden del tiempo de ejecución del algoritmo.
5. Ilustrar el funcionamiento del algoritmo buscando divisiones / departamentos que existan y que no existan.
6. Calcular la cantidad de comparaciones en cada caso.
7. Hallar el orden del tiempo de ejecución de cada algoritmo escrito.

Ejercicio 2

Se desea construir un índice de palabras de un libro. Para ello, recorriendo el texto del mismo se han de insertar las palabras en una estructura de trie, indicando, para cada palabra, la(s) páginas del libro en que se encuentra la misma. Utilizando lo desarrollado en el ejercicio domiciliario, el Equipo ha de consolidar una propuesta que tenga:

1. La descripción (esquema) de las estructuras de datos necesarias para construir un trie para el alfabeto compuesto por las letras a..z . Dibujarla en un POSTER.
2. Dibujo del trie correspondiente al siguiente conjunto de datos de prueba (POSTER):
 - Ala, 1, 3, 88
 - Alimaña, 11, 22
 - Alabastro, 4
 - Perro, 5, 8
 - Pera, 7,12
 - Alimento, 9
 - Casa, 11,13
 - Casada, 1

- Cazar, 33
 - Programa, 22, 67
 - Programación, 15
 - Programar 15,16
3. Un algoritmo para, a partir de un texto, construir el índice del mismo en el trie, indicando las páginas en que aparecen las palabras.
 4. Un algoritmo para, dado el trie, buscar una cadena y, si existe, devolver los números de páginas del libro en los que esa palabra se encuentra.

El Equipo realizará en conjunto las actividades 1 y 2. Se ha de separar en sub-equipos para desarrollar las actividades 3 y 4, cuyos resultados luego serán integrados.

Ejercicio 3

Escenario Se desea crear un programa que permita indizar todas las palabras almacenadas en múltiples textos digitales. Este programa ha de tener una funcionalidad de búsqueda de las palabras en forma parcial, incremental, similar a las búsquedas de google o al “autocompletar” de muchos editores de código fuente.

El analista del proyecto ha definido que la mejor forma de representar este problema es mediante el uso de un TRIE, dada la eficiencia del mismo para la búsqueda de strings y, particularmente, de prefijos, permitiendo una búsqueda de patrones incremental.

De las funcionalidades que debe tener el sistema a desarrollar, el analista ha asignado a su grupo de desarrolladores las siguientes:

- Insertar una nueva palabra en este diccionario.
- Buscar las ocurrencias de una cierta palabra.

Se requiere:

1. Implementar el método insertar de Trie.
2. Implementar el método buscar de Trie.
3. Definir los casos de prueba.

Ejercicio 4

Describir en lenguaje natural, pre y post condiciones, pseudocódigo de las operaciones y análisis de ordenes de las siguientes operaciones del TArbolGenerico y TNodeGenerico:

1. Agregar un hijo a un padre dado.

```
boolean agregarHijo(Comparable<T> padre, T hijo)
```

2. Eliminar los nodos que satisface un criterio.

```
void eliminar(Comparable<T> criterio)
```

3. Buscar un dato según un criterio de búsqueda.

```
T buscar(Comparable<T> criterio)
```

4. Obtener el grado de un nodo según un criterio de búsqueda.

```
int grado(Comparable<T> nodo)
```

Ejercicio 5

Describir en lenguaje natural, pre y post condiciones, pseudocódigo de las operaciones y análisis de ordenes de las siguientes operaciones del Trie y NodoTrie:

1. Operación buscar palabra completa.
2. Obtener la lista de palabras por un prefijo dado.
3. Insertar una palabra con un dato asociado.
4. Eliminar una palabra del Trie.

Ejercicio 6

Parte 1

Dado el siguiente conjunto de palabras: **if**, **int**, **for**, **static**, **class**, **new**, **this**, **add**.

Insertarlas en el orden dado en una tabla hash, usando:

1. sondeo lineal
2. sondeo cuadrático

Para cada caso, calcular el tamaño de tabla adecuado y usar una función sencilla, rápida y adecuada a la cantidad de claves.

Parte 2

Utilizando los conceptos manejados de TDA, es necesario implementar un THash en Java. Dadas las siguientes operaciones:

1. Devuelve la cantidad de comparaciones realizadas

```
public int buscar(int unaClave)
```

2. Devuelve la cantidad de comparaciones realizadas

```
public int insertar(int unaClave)
```

3. Devuelve la posición generada por la función

```
public int funcionHashing(int unaClave)
```

Se solicita:

1. Desarrollar, en pseudocódigo, las operaciones de inserción y búsqueda.

2. Definir el tamaño de la tabla de acuerdo con las mejores prácticas, considerando las claves del archivo `claves_insertar.txt`.
3. Diseñar una función hash eficiente (presentar también en pseudocódigo).
4. Implementar las operaciones de inserción y búsqueda en Java, registrando la cantidad de comparaciones efectuadas.
5. Preparar el programa para realizar inserciones y búsquedas masivas, utilizando los archivos `claves_insertar.txt` y `claves_buscar.txt`.

Consideraciones

- El método de resolución de colisiones a utilizar es el direccionamiento abierto lineal, con la fórmula: $h(i) = (h(0) + i) \bmod M$. Donde M es el tamaño de la tabla.
- El programa principal debe:
 - Crear la tabla hash.
 - Leer las claves del archivo de entrada.
 - Establecer el tamaño de la tabla como: $M = cantidadClaves/0,9$.
- Ejercutar las operaciones de inserción y búsqueda y mostrar los resultados.

Dinámica del trabajo

El grupo debe dividirse en dos subequipos.

Paso 1

- Subequipo A - desarrolla el pseudocódigo de las operaciones de inserción y búsqueda.
- Subequipo B - desarrolla el pseudocódigo de la función hash más eficiente posible.

Paso 2

- Intercambiar los pseudocódigos, realizar correcciones y elaborar un poster final con el diseño completo.

Parte 3

A partir de los pseudocódigos elaborados en el ejercicio anterior, implementar el programa completo utilizando el proyecto base publicado en la web.

Paso 1

- Subequipo A: implementa la función hash y la prueba con un conjunto pequeño de datos.
- Subequipo B: implementa las operaciones de inserción y búsqueda, y diseña el programa principal, que debe:
 - Leer las claves a insertar y registrarlas en la tabla.
 - Leer las claves a buscar y ejecutar las búsquedas.

- Mostrar la cantidad promedio de comparaciones realizadas para búsquedas exitosas e infructuosas.

Paso 2

- Integrar el código de ambos subequipos y realizar las pruebas con los archivos `claves_buscar.txt` y `claves_insertar.txt`.
- Reportar resultado de cada búsqueda (éxito o no encontrado).

Ejercicio 7

Parte 1

Instrucciones de uso de las clases para medición.

Parte 2

Dado el conjunto de palabras «listado-general-desordenado.txt» cargarlas en las siguientes estructuras de datos:

- `java.util.LinkedList`
- `java.util.ArrayList`
- `TTrie`
- `java.util.HashMap`
- `java.util.TreeMap`

Parte 3

Cargar en memoria las claves «listado-general-palabrasBuscar.txt» y ejecutar las búsquedas 20 veces.

Para cada estructura cargada, implementar la clase de medición. Se provee como ejemplo la correspondiente a `LinkedList`. Implementar para:

- `ArrayList`
- `TTrie`
- `HashMap`
- `TreeMap`

Parte 4

Consolidar los resultados en una tabla que contenga, para cada estructura utilizada, el consumo de memoria (en MBs) y el tiempo de ejecución (en milisegundos).

Parte 5

Se desea ahora comparar diferentes alternativas para implementar el programa desarrollado en la unidad para predecir o autocompletar palabras. Queremos evaluar implementaciones utilizando:

- Trie
- LinkedList
- HashMap

Dado el conjunto de palabras «listado-general-desordenado.txt», genera las clases de medición para cada implementación de «predecir» en forma análoga a las clases de medición usadas en el ejercicio anterior.

Efectuar 20 mediciones de cada alternativa para predecir todas las palabras contenidas que comiencen con «cas».

Consolidar los resultados en una tabla que contenga, para cada estructura utilizada, el consumo de memoria (en MBs) y el tiempo de ejecución (en milisegundos).

Ejercicio 8

En el trie desarrollado anteriormente aprovechamos el conocimiento de los códigos ASCII del conjunto de caracteres del alfabeto para acceder al hijo asociado. Luego desarrollamos un pequeño programa que tiene la funcionalidad de «autocompletar» o «predecir» texto a medida que vamos tecleando. Sin embargo, esta implementación resulta muy poco flexible si queremos extenderla para otros alfabetos. Una forma de resolver esto sería utilizando, dentro del TNodeTrie, un HashMap que, dada una clave, devuelva la referencia al TNodeTrie correspondiente (subárbol).

Se requiere entonces:

- Desarrolla nuevamente el trie utilizando HashMap en el TNodeTrie.
- Inserta en el trie las strings indicadas y prueba el programa.
- Compara las 2 implementaciones, en cuanto a consumo de memoria y tiempo de ejecución (para esto último será necesario crear un método que ejecute múltiples búsquedas, muchas veces, y medir el tiempo utilizando las funciones de java).

Ejercicio 9

Utilizando como base el programa desarrollado en el «Trabajo de Aplicación 1», Ejercicio 3, se desea hacer una evaluación de la evolución del rendimiento de la tabla hash en las operaciones de inserción y búsqueda, a medida que se va incrementando el factor de carga.

Para ello, se deberá realizar una planilla o tabla que contenga 4 columnas:

1. Factor de carga (empezar en 70 %, incrementando en 5 % hasta el 90 %, y luego incrementos del 1 % hasta llegar al 99 %).
2. Promedio de comparaciones realizadas para insertar una clave.
3. Promedio de comparaciones realizadas en las búsquedas exitosas.
4. Promedio de comparaciones realizadas en las búsquedas sin éxito.

Factor de carga %	Prom.Comp. Inserción	Prom. Comp. Búsqueda existosa.
70		
75		
80		
85		
90		
91		
92		
93		
94		
95		
96		
97		
98		
99		

Se sugiere utilizar una planilla electrónica para registrar los resultados. Graficar los resultados.

¿Se condicen estos resultados con los conceptos vistos en forma teórica?

Ejercicio 10

Parte 1

Escribe, en el menor número de líneas de código posible, una funcionalidad que elimine todas las entradas de un Map cuyo valor sea null.

Parte 2

Escribe un método que tome como parámetro un Map<String,String>y devuelva un nuevo Map<String, String>en el que las claves y los valores estén intercambiados. Genera una excepción si hay valores duplicados en el mapa que se pasa como parámetro.

Parte 3

Escribe un programa que lea cadenas de caracteres de entrada y las imprima ordenadas según su longitud, comenzando por la más corta. Si hay varias cadenas con la misma longitud, éstas deben imprimirse en orden lexicográfico.

Parte 4

Escribe un programa que lea un archivo de texto, pasado como primer parámetro en la línea de comando, en una List. El programa debe luego imprimir en forma aleatoria líneas del archivo. El número de líneas a imprimir será especificado como segundo parámetro. Escribe este programa de forma tal que asigne una colección del tamaño correcto desde el inicio, y no una que vaya expandiéndose gradualmente a medida que el archive es leído. Para calcular la cantidad de líneas del archive se puede usar la función *java.io.File.length* para obtener el tamaño del archivo y luego dividir por el tamaño considerado de una línea promedio.

Ejercicios del Documento IBM Java Collections Framework

Utilizando el documento disponible en la web [http://www.ibm.com/java/collections-framework.html](#), se recomienda realizar los siguientes ejercicios, que son todas implementaciones muy

utilizadas habitualmente en programas reales:

- Exercise 2. «How to use a TreeSet to provide a sorted JList», pág. 38.
- Exercise 3. «How to use an ArrayList with a JComboBox», pág. 40.
- Exercise 4. «How to use a map to count words», pág. 41.

Ejercicio 11

Se desea implementar un programa que, en forma eficiente, permita obtener las frecuencias de ocurrencias de las palabras de un libro. Para ello se debe entonces:

1. Seleccionar la o las clases de la API de colecciones de JAVA más apropiada para esta aplicación. Justificar la elección.
2. Dado un archivo de entrada «libro.txt», desarrollar un programa JAVA utilizando la API de colecciones que permita saber las frecuencias de ocurrencias de las palabras del libro.
3. Graficar los resultados de las 10 palabras que más ocurren.

Ejercicio 12

Ahora que conocemos la librería de colecciones de JAVA, deseamos mejorar nuestra implementación de Tries, para hacerla más flexible y fácilmente adaptable a diferentes alfabetos y contextos.

Se propone entonces utilizar, como forma de almacenamiento de los enlaces a los subárboles en los nodos del Trie, una estructura de HashMap. Utilizando entonces los ejemplos y código de la unidad, desarrolla:

- Nuevas clases TTrieHashMap y TNodeTrieHashMap (ésta última utiliza para la representación un hashMap, en vez del vector fijo).
- Adapta los métodos de inserción, búsqueda, búsqueda de prefijos (“predecir”) y búsqueda de patrones (vistos en la aplicación de tries como árboles de sufijos).
- Implementa nuevamente la aplicación de “autocompletar”, utilizando estas nuevas clases, y haz las pruebas que estimes convenientes.
- Implementa también la aplicación para la búsqueda de patrones en un cierto texto, indicando las posiciones del texto en que ocurren.

Ejercicio 13

Parte 1

Investiga cómo está implementado el método hashCode en Java para objetos de la clase Object. Luego investiga cómo se implementa el mismo para las clases Integer y String. Explica por qué la implementación es diferente.

Parte 2

Investiga y diagrama cómo son las estructuras internas de un HashMap. Con lo investigado en el ejercicio anterior, diagrama el estado de las estructuras luego de insertar las siguientes strings:

- Hola
- HolaMundo
- HashMap
- Colecciones

Parte 3

Implementa los métodos equals y hashCode para la siguiente clase:

```
public class Alumno{
    private int id;
    private String fullName;
    private String email;

    // ....
}
```

Indica qué características deben tener las implementaciones de los métodos solicitados para mantener el contrato general para el método hashCode.

Ejercicio 14

Dado un mismo conjunto de claves enteras, insertar las claves en tablas hash usando diferentes estrategias de resolución de colisiones:

- Sondeo lineal.
- Sondeo cuadrático.
- Encadenamiento separado.

Utilizar el siguiente conjunto de claves, en el orden indicado:

45, 12, 37, 82, 29, 54, 31, 76, 18, 93, 11, 68

Se pide:

1. Definir un tamaño de tabla adecuado y justificar la decisión.
2. Definir una función hash sencilla para las claves dadas.
3. Mostrar el estado final de la tabla para cada estrategia.
4. Calcular la cantidad total de colisiones en cada caso.
5. Calcular el promedio de comparaciones para búsquedas exitosas.
6. Calcular el promedio de comparaciones para búsquedas no exitosas, indicando claramente qué claves se utilizaron para probarlas.
7. Comparar los resultados obtenidos y concluir cuál estrategia funcionó mejor para este conjunto de datos.

Ejercicio 15

Dada la siguiente clase Java:

```
public class Libro {  
    private String isbn;  
    private String titulo;  
    private String autor;  
    private int anio;  
  
    // constructores, getters y setters  
}
```

Se desea utilizar objetos de tipo `Libro` dentro de estructuras basadas en hashing, como `HashSet` y `HashMap`.

Se pide:

1. Definir cuál o cuáles atributos deberían determinar la identidad lógica de un libro. Justificar la respuesta.
2. Implementar los métodos `equals` y `hashCode` respetando el contrato general de Java.
3. Explicar qué error conceptual aparece si `equals` compara libros usando `isbn`, pero `hashCode` se calcula usando `titulo`.
4. Construir un ejemplo en el que dos objetos `Libro` distintos representen el mismo libro y sean insertados en un `HashSet`.
5. Explicar qué debería ocurrir en ese `HashSet` si `equals` y `hashCode` están correctamente implementados.
6. Proponer casos de prueba para verificar que la implementación cumple el contrato de `equals` y `hashCode`.

Ejercicio 16

Se desea modelar un árbol genealógico, donde cada nodo representa una persona. Implemente una estructura de datos que permita representar una familia a lo largo de varias generaciones. Cada persona debe tener al menos la siguiente información: Nombre y Año de nacimiento. Las relaciones familiares se modelan de la siguiente forma:

- Cada nodo puede tener cero o más hijos
- No se consideran padres explícitos (el árbol se construye desde un ancestro común)

Parte 1: Construcción

Definir la clase Persona. Definir la estructura de nodo para el árbol genérico. Construir manualmente un árbol con al menos: 3 generaciones 10 personas en total

Ejemplo conceptual (no hace falta que lo representen así):

```
Abuela
|-- Hijo1
|   |-- Nieto1
|   |-- Nieto2
|-- Hijo2
    |-- Nieto3
```

Parte 2: Implementar los siguientes métodos

1. Listar todos los descendientes de una persona dada
2. Calcular la altura del árbol
3. Contar la cantidad total de personas
4. Obtener todas las personas de una generación dada (por ejemplo: generación 0 = raíz, generación 1 = hijos, etc.)
5. Encontrar el ancestro común más cercano entre dos personas
6. Determinar si una persona es descendiente de otra

Ejercicio 17

Parte 1:

Investigar brevemente qué es una función de hash criptográfica y cuáles son sus propiedades.

Buscar una implementación de **SHA-256** (puede ser de librería estándar o externa). Implementar una función: `byte[] calcularHash(byte[] archivo)` que:

1. reciba el contenido de un archivo como array de bytes
2. devuelva el hash **SHA-256** correspondiente

Parte 2:

Dados dos archivos, uno original y otro con una modificación mínima (por ejemplo, cambiar un solo byte) Responder:

1. ¿Cuál es la probabilidad de que dados dos archivos diferentes colisionen?
2. ¿Qué herramienta que usan a diario (o deberían), creen que usa esto? ¿Para qué?